

Buscando la “Joya Oculta”: Comparativa de Sistemas de Recuperación de Información en un Corpus de Películas

Sandra Silva Amaya

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
sansilama@alum.us.es

Nuria Martín Sánchez

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
nurmarsan@alum.us.es

Resumen—El objetivo de este trabajo es desarrollar y comparar dos sistemas de recuperación de información. El primero de ellos se basa en el modelo de recuperación booleano, mientras que el segundo admite consultas de texto libre, empleando el modelo de ponderación TF-IDF y la similitud del coseno para devolver una lista ordenada de resultados. Para implementar ambos sistemas, se llevó a cabo un preprocesamiento del lenguaje natural sobre el corpus de prueba mediante tokenización, eliminación de palabras vacías (*stop words*) y extracción de raíces (*stemming*), apoyándose en el uso de librerías como NLTK y Whoosh.

La evaluación de ambos sistemas se realizó utilizando métricas de precisión y exhaustividad (*recall*). El sistema booleano alcanzó una precisión de 0.998 y un *recall* de 1.0, mientras que el modelo TF-IDF obtuvo un Mean Average Precision (MAP) de 0.7248. No obstante, se determinó que estos resultados no implican necesariamente una superioridad del modelo booleano en escenarios reales. Debido a que el corpus de prueba y las necesidades de información fueron diseñados por las mismas autoras del proyecto, las consultas booleanas se beneficiaron de un conocimiento previo del vocabulario indexado. Teniendo en cuenta este sesgo conceptual, se concluye que el modelo TF-IDF presentaría un rendimiento más robusto y realista frente a una colección de prueba evaluada por usuarios externos que no conozcan de antemano el contenido exacto de los documentos.

Palabras clave—Recuperación de información, modelo booleano, TF-IDF, procesamiento del lenguaje natural, evaluación de sistemas de búsqueda

I. INTRODUCCIÓN

Como campo académico de estudio, la recuperación de información (RI) se define como el proceso de buscar y encontrar material de naturaleza no estructurada, predominantemente texto, que satisfaga una necesidad de información dentro de grandes colecciones de documentos [1]. En la actualidad, esta disciplina ha superado con creces los entornos tradicionales de bases de datos relacionales, convirtiéndose en la tecnología subyacente fundamental en motores de búsqueda web, sistemas de correo electrónico y plataformas de recomendación de contenido multimedia [2]. Para gestionar este volumen de información textual, se han desarrollado históricamente diversos modelos matemáticos de recuperación, los cuales varían en complejidad y enfoque lógico, abarcando desde las

coincidencias lógicas exactas hasta los sistemas basados en espacios vectoriales.

El lenguaje utilizado en las reseñas de películas y sinopsis suele ser muy libre, lo que introduce variaciones lingüísticas que afectan a la recuperación de información. Para unificar el texto antes de generar el índice, se aplica una etapa de preprocesamiento de lenguaje natural (PLN) basada en tokenización, eliminación de palabras vacías (*stop words*) y reducción morfológica (*stemming*) [3]. Con el corpus ya estructurado, el comportamiento de los motores de búsqueda se valida empleando las métricas habituales del sector: precisión, *recall* y el promedio de precisión media (MAP).

El objetivo principal de este trabajo es el diseño, la implementación y la evaluación comparativa de dos aproximaciones clásicas en la recuperación de información. El primer sistema propuesto se fundamenta en el modelo de recuperación Booleano, implementado mediante la biblioteca Whoosh, el cual opera bajo una lógica de correspondencia estricta. El segundo sistema consiste en un motor de consultas en texto libre utilizando la ponderación TF-IDF y el cálculo de la similitud del coseno para establecer un ranking ordenado de relevancia. Ambos enfoques son validados empleando un corpus propio de sinopsis y reseñas cinematográficas, confeccionado junto con un banco de veinte necesidades de información y sus respectivos juicios de relevancia. A través de este escenario, se persigue analizar las limitaciones prácticas de ambos modelos, prestando especial atención a fenómenos como el sobreajuste y el sesgo de autoría en la evaluación.

El resto de este documento se estructura de la siguiente manera: la Sección II detalla las características del corpus cinematográfico y los procesos de preprocesamiento de lenguaje natural aplicados; la Sección III describe la arquitectura matemática y el desarrollo de los modelos de recuperación Booleano y Vectorial; la Sección IV expone la metodología de evaluación y analiza comparativamente las métricas obtenidas por ambos sistemas. Finalmente, la Sección V recoge las conclusiones principales derivadas de este estudio y propone posibles líneas de trabajo futuro.

II. PRELIMINARES

En esta sección se introduce el marco teórico y tecnológico sobre el que se asientan los métodos aplicados en el presente estudio, así como una breve revisión de la literatura y trabajos previos directamente relacionados con la materia.

A. Métodos empleados

Para lograr el tratamiento automático del texto y permitir la recuperación efectiva de documentos, este trabajo se fundamenta en un conjunto de técnicas propias de la disciplina, las cuales abarcan desde la fase previa de limpieza del texto hasta los algoritmos subyacentes de emparejamiento.

- **Procesamiento de Lenguaje Natural (PLN):** Comprende las técnicas de normalización léxica previas a la indexación. Estas incluyen la tokenización (separación del texto en unidades mínimas), la eliminación de palabras vacías o *stop words* (términos funcionales como preposiciones o artículos carentes de significado semántico por sí solos) y la reducción morfológica o *stemming* [3]. El *stemming* es especialmente relevante en idiomas con alta flexión como el español, ya que reduce las palabras a su raíz común, minimizando el vocabulario e incrementando la probabilidad de coincidencia entre la consulta del usuario y el documento.
- **Modelo Booleano:** Consiste en el paradigma más clásico de recuperación de información, operando bajo un mecanismo de coincidencia exacta regido por el álgebra de Boole y la teoría de conjuntos. Las consultas se modelan utilizando operadores lógicos (*AND*, *OR*, *NOT*) sobre índices invertidos. Su principal ventaja es el control estricto que ofrece al usuario sobre los resultados, aunque carece de un sistema de ordenación por relevancia, ofreciendo un conjunto plano de documentos recuperados [1].
- **Modelo Vectorial (TF-IDF y Similitud del Coseno):** Se trata de un enfoque algebraico que representa tanto los documentos como la consulta en forma de vectores en un espacio de características multidimensional [1]. La ponderación TF-IDF (*Term Frequency - Inverse Document Frequency*) se emplea para asignar un peso a cada término, valorando positivamente su frecuencia dentro de un documento (TF) pero penalizándolo si aparece en la mayoría de los documentos del corpus (IDF), de modo que se destacan los términos verdaderamente discriminadores. Posteriormente, el cálculo de la similitud del coseno entre el vector de la consulta y el de cada documento permite determinar el ángulo que los separa, estableciendo así un nivel de relevancia continuo que facilita el retorno de un listado ordenado o ranking.

B. Trabajo Relacionado

La evaluación de sistemas de recuperación de información tiene una trayectoria muy consolidada en la literatura. Desde los experimentos clásicos de Cranfield hasta las conferencias TREC (*Text REtrieval Conference*), el estándar metodológico siempre se ha apoyado en la misma idea: medir el rendimiento combinando una colección de documentos, un conjunto de

necesidades de información y sus correspondientes juicios de relevancia [1].

Cuando nos centramos en el idioma español, diversos estudios señalan que las búsquedas literales se quedan cortas debido a la rica flexión morfológica del lenguaje. En este escenario, la literatura demuestra que aplicar algoritmos de extracción de raíces, como el *Snowball stemmer*, es una solución clave para agrupar variantes de una misma palabra y mejorar drásticamente la exhaustividad o *recall* del sistema.

Por otro lado, el uso de bases de datos de cine extraídas de Kaggle, como *TMDB* e *IMDB* [4], [5], es un recurso super explotado en nuestro campo. Sin embargo, casi todos los trabajos previos utilizan estos datasets para entrenar sistemas de recomendación (como filtrado colaborativo) o para predecir valoraciones. Lo que diferencia a nuestro proyecto es que cambiamos ese enfoque por completo: tomamos estos datos públicos de Kaggle para centrarnos estrictamente en la recuperación de texto. Esto nos ha permitido diseñar un banco de consultas, adaptando las metodologías clásicas de evaluación a un vocabulario y un contexto de usuario totalmente actuales.

III. METODOLOGÍA

En esta sección se detalla el proceso técnico y metodológico llevado a cabo para el diseño, desarrollo y evaluación de los sistemas de recuperación de información. El flujo de trabajo descrito se divide en cinco etapas fundamentales: la construcción de un corpus documental a medida, el establecimiento de un marco de pruebas basado en necesidades de información reales, la aplicación de técnicas de preprocesamiento de lenguaje natural para normalizar los textos, la indexación de los datos estructurados y, por último, la implementación algorítmica de los modelos de recuperación booleano y vectorial.

A. Construcción del Corpus Documental

Para satisfacer los objetivos del trabajo, se ha desarrollado un corpus documental propio. Dicha colección ha sido confeccionada a partir de la integración y limpieza de dos conjuntos de datos públicos e independientes extraídos de la plataforma Kaggle. En primer lugar, se utilizó el *TMDB 5000 Movie Dataset* [4] para extraer los metadatos estructurales de las películas, tales como el título, género, idioma y la sinopsis oficial. En segundo lugar, se integró el *IMDB Dataset* [5] para añadir al corpus texto subjetivo mediante reseñas de usuarios.

Dada la cantidad de los registros en los repositorios originales, la construcción del archivo estructurado final (CSV) requirió un proceso de selección manual. Se extrajo un subconjunto más reducido y representativo de películas, lo cual garantizó un entorno de datos manejable y ajustado al alcance de la evaluación. Asimismo, durante esta fase de unificación, se identificó la necesidad de adaptar gran parte de la información al idioma principal del proyecto (español). Para solventarlo, se empleó el modelo de lenguaje Gemini, desarrollado por Google [6], el cual sirvió como herramienta de asistencia para la traducción de los textos y la posterior unificación de los datos en el archivo CSV final.

B. Desarrollo de las Necesidades de Información

El proceso de evaluación de cualquier sistema de búsqueda requiere la definición de un marco de pruebas estandarizado que emule las intenciones de búsqueda de usuarios reales. Para cumplir con esta metodología, se diseñaron tres componentes fundamentales: el banco de necesidades de información (NI), el conjunto de consultas asociadas y los juicios de relevancia correspondientes.

En primer lugar, se definieron manualmente exactamente 20 necesidades de información. Al formular estas NI, se procuró ir más allá de simples búsquedas por palabras clave aisladas, redactándolas como oraciones completas en lenguaje natural que combinan criterios objetivos (idioma, género cinematográfico) con valoraciones cualitativas y subjetivas extraídas de las críticas de los usuarios (tales como la predictibilidad de la trama, o si se considera una “joya oculta” o una “decepción”). Esta diversidad conceptual permite someter a ambos motores de búsqueda a un espectro amplio de escenarios lógicos y léxicos.

En segundo lugar, se generó el archivo `consultas.csv` para formalizar cómo cada NI se traslada a los sistemas de recuperación de información. Para cada necesidad, se crearon dos representaciones diferenciadas:

- **Consulta Booleana:** Una expresión formal estructurada utilizando los operadores booleanos AND, OR y NOT, junto con el uso de paréntesis para definir la precedencia lógica del filtrado (e.g., `fantasía AND (familia OR animación)`). Estas consultas están diseñadas para interactuar directamente con la sintaxis del parser de la biblioteca *Whoosh*.
- **Consulta de Texto Libre:** Una cadena de texto libre desprovista de operadores lógicos (e.g., `fantasía familiar animación`), que emula la consulta que un usuario escribiría en un buscador moderno. Estas cadenas son procesadas por el motor vectorial basado en TF-IDF y similitud del coseno.

Finalmente, para poder calcular métricas de efectividad de forma cuantitativa, se construyó el archivo `relevancia.csv`, que constituye la verdad de referencia del sistema. En lugar de realizar este etiquetado de forma manual, el proceso se automatizó mediante inteligencia artificial utilizando el modelo de lenguaje Gemini de Google [6], el cual evaluó si el contenido de cada una de las 70 películas (su sinopsis, idioma, reseñas de usuarios y géneros) satisfacía semánticamente cada una de las 20 necesidades de información. Esta evaluación dio como resultado la anotación de un juicio binario de relevancia (donde 1 indica que el documento es relevante para la necesidad de información dada y 0 que no lo es). Este proceso de etiquetado se rigió por criterios estrictos basados en los metadatos de las obras. Por ejemplo, ante una NI que solicitaba películas en “español”, un documento solo fue marcado como relevante en `relevancia.csv` si su idioma oficial era estrictamente “Español”. Cabe resaltar que esto difiere del comportamiento del algoritmo de búsqueda implementado, dado que los motores indexan un campo de texto

combinado (`contenido`), si una película en inglés contiene el término “español” en su sinopsis o reseña (por ejemplo, aludiendo a un personaje o lugar), el algoritmo la recuperará de todos modos. De esta forma, `relevancia.csv` mantiene una exigencia de veracidad estructural frente a la cual se evalúa la precisión y recall de los modelos.

C. Preprocesamiento de Lenguaje Natural

Con el fin de optimizar el rendimiento de los motores de búsqueda, el corpus textual se sometió a una fase de preprocesamiento de lenguaje natural (PLN). Esto era necesario pues al tratarse de un conjunto de películas diversas con reseñas escritas por múltiples usuarios, la variabilidad lingüística era muy alta, lo que dificultaría a los modelos de recuperación y perjudicaría la coincidencia de términos.

La limpieza se estructuró en varias etapas consecutivas utilizando la librería *NLTK*. En primer lugar, el texto se convirtió a minúsculas y se aplicó una tokenización para fragmentar las cadenas en palabras individuales. Posteriormente, mediante expresiones regulares, se eliminaron los caracteres no alfanuméricos y los signos de puntuación. Para reducir el tamaño del vocabulario y evitar términos irrelevantes, se filtraron las palabras vacías (*stop words*) empleando el diccionario en español de la librería. Finalmente, se aplicó el algoritmo *SnowballStemmer* para reducir las palabras restantes a su raíz morfológica común. De este modo, se garantiza que términos con la misma base semántica se agrupen bajo un único identificador durante la indexación y la vectorización.

D. Indexación

Una vez normalizado el texto, los datos se estructuraron combinando los campos en un documento indexable. Para ello, hicimos uso de la biblioteca *Whoosh*, la cual nos permitió generar un índice invertido persistente en disco de forma eficaz.

El proceso de construcción del índice se dividió en dos fases principales. En primer lugar, se definió un esquema lógico (*Schema*) para modelar la estructura de cada película. A cada película se le asignó un identificador único (ID) y varios campos de texto (`TEXT`) para almacenar sus metadatos (título, idioma, sinopsis, reseña y género). Todos estos atributos se configuraron con el parámetro `stored=True`, lo cual es fundamental para que el sistema no solo busque, sino que pueda recuperar y mostrar esa información al usuario en la interfaz final. Adicionalmente, se introdujo un campo clave denominado `contenido`, destinado a agrupar la versión preprocesada de los textos y sobre el cual se realizan las búsquedas globales.

En la segunda fase, se configuró un directorio local (`indexdir`) donde el sistema aloja físicamente el índice invertido. Mediante la apertura de un escritor (*IndexWriter*), se iteró sobre la estructura de datos que contenía el corpus (*DataFrame*), añadiendo secuencialmente cada película como un nuevo documento. Finalmente, se consolidaron los cambios en memoria mediante una operación de *commit*, dejando el

índice cerrado, listo para ser utilizado por los motores de búsqueda.

E. Modelos de Recuperación

Para la implementación técnica, el sistema se dividió en dos motores:

- 1) **Motor Booleano:** Este sistema evalúa consultas estructuradas por el usuario mediante operadores lógicos (*AND*, *OR* y *NOT*) sobre el índice invertido generado previamente, recuperando únicamente las películas que cumplen de forma estricta con dichas restricciones. El motor está construido con el soporte de la biblioteca *Whoosh*, haciendo uso de su analizador sintáctico (*QueryParser*). Esta herramienta permite interpretar la consulta de manera robusta, gestionando correctamente la jerarquía de los paréntesis y aislando los espacios en blanco para evitar errores de sintaxis. Asimismo, el sistema separa los operadores lógicos y aplica de forma automática la misma canalización de preprocesamiento (conversión a minúsculas, filtrado y *stemming*) a los términos de búsqueda individuales, garantizando así una coincidencia exacta con el vocabulario ya indexado.
- 2) **Motor de Espacio Vectorial:** Este modelo está implementado con la ayuda de *scikit-learn* y funciona siguiendo unos pasos muy directos. Cuando el usuario introduce una consulta, lo primero que hace el sistema es procesar y limpiar ese texto. Después, utiliza la herramienta *TfidfVectorizer* para transformar tanto los textos de las películas como la consulta en vectores. Una vez que tenemos la información convertida a pesos TF-IDF, calculamos la similitud del coseno entre la búsqueda y cada película del corpus. Esto nos devuelve una lista de puntuaciones que ordenamos de mayor a menor para que los resultados más relevantes aparezcan primero. Por último, el código aplica un filtro muy sencillo: solo guarda y devuelve los identificadores de aquellas películas cuya similitud sea estrictamente mayor a 0.00.

F. Decisiones de Diseño

En primer lugar, una de las decisiones fundamentales que guió el desarrollo de este trabajo fue la selección y confección manual del corpus documental. Tras analizar diversas colecciones de prueba públicas preexistentes, se determinó que sus estructuras no resultaban lo suficientemente atractivas o no se ajustaban por completo a los objetivos del proyecto. Por este motivo, se optó por construir un corpus a medida centrado exclusivamente en la temática del cine. Esta elección respondió a un doble criterio: es un ámbito culturalmente interesante y ambas autoras cuentan con conocimiento sobre dicho tema.

A nivel técnico, una de las decisiones arquitectónicas clave en el diseño del esquema de indexación fue el desacoplamiento entre los campos destinados estrictamente a la búsqueda y los campos utilizados para la visualización y almacenamiento de datos. En particular, se optó por indexar por separado los atributos de metadatos (*título*, *idioma*, *género*) y los textos descriptivos (*sinopsis*, *reseña*), configurando el motor de

búsqueda para almacenar el contenido original de todos ellos, de forma complementaria al campo combinado *contenido*.

Esta elección de diseño responde a dos necesidades fundamentales:

- 1) **Eficacia en la Recuperación:** Si las búsquedas o los cálculos de similitud se realizaran directamente sobre los textos originales (sin preprocesar, con alta variabilidad, tildes y mayúsculas), el rendimiento de ambos modelos se vería severamente afectado. Por ello, el campo *contenido* almacena exclusivamente el texto unificado y normalizado tras pasar por la canalización de preprocesamiento (conversión a minúsculas, filtrado y *stemming*).
- 2) **Experiencia de Usuario e Interacción:** Cuando un motor de búsqueda recupera un documento, no es viable mostrar al usuario el texto preprocesado del campo *contenido* (e.g., “*avat espanol explor...*”), ya que resultaría ilegible. Al guardar las versiones originales de la sinopsis, el título o la reseña en el almacenamiento del índice, el sistema puede recuperar de manera instantánea la información legible original una vez identificado el ID de la película coincidente.

Derivado de esta separación, se ha implementado una mejora interactiva en la interfaz de usuario de consola. Tras realizar una búsqueda (ya sea mediante coincidencia booleana o por similitud de coseno en el modelo vectorial), el sistema presenta al usuario un listado numerado con las películas recuperadas. Aprovechando que la sinopsis y las reseñas se encuentran almacenadas directamente en el índice, el usuario puede seleccionar el número de una película específica para desplegar de forma inmediata su ficha detallada en pantalla (título original, géneros, idioma, sinopsis y reseñas completas). Este diseño evita tener que realizar lecturas recurrentes al archivo físico de la base de datos (*corpus.csv*) en disco, manteniendo el sistema autocontenido y optimizando los tiempos de respuesta.

Adicionalmente, se tomó una decisión clave respecto al comportamiento del motor de recuperación basado en el modelo vectorial. Al emplear el cálculo de la similitud del coseno entre las consultas y el corpus, se implementó un filtro de exclusión estricto sobre las puntuaciones obtenidas. El sistema descarta automáticamente y no muestra aquellas películas cuya similitud con la consulta sea exactamente igual a 0.00. El propósito de esta medida es evitar la presentación de resultados nulos al usuario, garantizando así que los documentos devueltos en el ranking contengan, como mínimo, alguna coincidencia real con los términos de búsqueda solicitados.

Por otra parte, a nivel de ingeniería de software, se optó por estructurar el proyecto mediante una separación de archivos diferenciados según su propósito, dividiéndose en dos cuadernos interactivos y dos módulos en formato script. Los cuadernos, orientados al desarrollo iterativo y la documentación visual, se utilizaron para las tareas que requieren una verificación paso a paso: el primero (*procesar_corpus.ipynb*) para la limpieza de los metadatos y la generación del índice, y el segundo (*evaluar_sistemas.ipynb*) para la eje-

cución estadística de la evaluación y la obtención de métricas agregadas. Por el contrario, la lógica de preprocesamiento y búsqueda se encapsuló en un módulo de utilidades independiente (*funciones.py*) para permitir que tanto ambos cuadernos de trabajo como la interfaz final reutilicen las mismas funciones sin duplicidad de código, garantizando la consistencia total en el tratamiento del texto. Finalmente, la interfaz de usuario se implementó en un script independiente (*interfaz.py*), el cual permite su ejecución rápida y directa desde la terminal del sistema operativo, asegurando una experiencia de uso ágil y desacoplada de la fase de desarrollo.

G. Uso de Inteligencia Artificial Generativa

Siguiendo las directrices de la asignatura, el uso de sistemas de inteligencia artificial generativa se limitó a tareas de asistencia en el procesamiento de datos y etiquetado de relevancia, utilizando el modelo de lenguaje Gemini desarrollado por Google [6]. A continuación se describen los propósitos y las entradas (*prompts*) empleadas en cada caso:

- **Traducción y Unificación del Corpus:** Para traducir las sinopsis y reseñas cinematográficas del inglés al español, se proporcionó al modelo un *prompt* estructurado donde se definía su rol como traductor profesional. Se le instruyó para mantener un tono natural, respetar nombres propios y marcas temporales, y devolver exclusivamente el texto traducido limpio de metadatos adicionales.
- **Generación de Juicios de Relevancia:** Para construir el archivo de verdad de referencia (*relevancia.csv*), se diseñó un *prompt* que incluía la descripción detallada de una necesidad de información (NI) y la ficha técnica de una película. Se solicitó al modelo analizar semánticamente el documento y responder con un valor binario (1 para relevante y 0 para no relevante), acompañado de una breve justificación de su lógica para validar la coherencia conceptual del etiquetado.

IV. RESULTADOS

En esta sección se dejan asentados los experimentos llevados a cabo para evaluar el rendimiento de los dos sistemas de recuperación de información desarrollados: el modelo booleano (basado en la biblioteca Whoosh) y el modelo vectorial (basado en la ponderación TF-IDF y la similitud del coseno). Primero se describe el banco de datos utilizado para los experimentos, seguido del análisis de los resultados cuantitativos obtenidos por ambos enfoques, y se concluye con un análisis comparativo de sus fortalezas y limitaciones.

A. Diseño Experimental y Configuración

Para evaluar la efectividad de ambos motores de búsqueda, se confeccionó un banco de pruebas compuesto por:

- **Colección de Documentos (Corpus):** Un total de 70 películas con información estructurada que integra título, idioma, sinopsis, géneros y reseñas de usuarios. Tras el preprocesamiento (conversión a minúsculas, tokenización, eliminación de caracteres no alfanuméricos, eliminación de palabras vacías en español y reducción

a raíces léxicas mediante Snowball), cada película se indexó como un único documento consolidado.

- **Necesidades de Información (NI):** Se definieron manualmente 20 necesidades de información representativas de búsquedas reales de usuarios, abarcando desde filtros temáticos directos (género, idioma) hasta criterios más subjetivos extraídos de las críticas (como “joya oculta” u “obra maestra”).
- **Consultas Formuladas:** Para cada NI, se derivaron dos consultas específicas: una expresión booleana formal (utilizando operadores AND, OR, y NOT) y una consulta en texto libre para el motor vectorial.
- **Juicios de Relevancia (Verdad de Referencia):** Se generó un conjunto binario de relevancia (1 para relevante, 0 para no relevante) mediante la asistencia del modelo de lenguaje Gemini de Google [6], el cual evaluó cada una de las 70 películas frente a las 20 necesidades de información para calcular las métricas de efectividad.

Para el sistema booleano, al tratarse de un modelo no posicionado (que devuelve un conjunto desordenado de documentos), se utilizaron las métricas de **Precisión** y **Sensibilidad o Recall** promediadas sobre las 20 consultas. Para el sistema vectorial, dado que devuelve una lista ordenada de documentos según su similitud del coseno, se evaluó mediante la **Precisión Media (Average Precision, AP)** de cada consulta y el **Promedio de Precisión Media (Mean Average Precision, MAP)** global del sistema.

B. Evaluación del Sistema Booleano

Los resultados promediados obtenidos por el motor booleano muestran una **Precisión Promedio de 0.9980** (99.8%) y un **Recall Promedio de 1.0000** (100%).

Un desglose detallado de los resultados por consulta (véase la Tabla I) revela que en 19 de las 20 necesidades de información el sistema booleano logró una precisión de 1.0 y un recall de 1.0. La única desviación se dio en la necesidad de información número 6 (“*Que sea de acción en español y no sea predecible*”), donde la precisión disminuyó ligeramente a 0.96 debido a la presencia de un único falso positivo. Específicamente, la película *The Hobbit: The Desolation of Smaug* fue recuperada por el motor booleano. Aunque esta obra no cuenta de forma explícita con el género de “Acción” en sus metadatos oficiales, en el texto de su sinopsis se describe como una película “cargada de acción”. Dado que el sistema indexa y busca sobre el campo unificado de contenido, el término “acción” en la sinopsis cumplió con la condición lógica de la consulta, provocando que se clasificara en los resultados. El recall se mantuvo en 1.0 para la totalidad de las consultas.

Estos números tan elevados están condicionados por el **sesgo de autoría (author bias)**. Puesto que las mismas personas diseñaron el corpus, las necesidades de información y las consultas booleanas, se produjo un acoplamiento cognitivo. Al redactar las consultas booleanas, las autoras emplearon términos exactos sabiendo de antemano el vocabulario indexado en los documentos relevantes. En un entorno real

con usuarios externos, el rendimiento del modelo booleano disminuiría notablemente debido a problemas de sinonimia o polisemia.

C. Evaluación del Sistema Vectorial (TF-IDF y Coseno)

El sistema de espacio vectorial obtuvo un **Mean Average Precision (MAP) de 0.7248** (72.48%) sobre las 20 necesidades de información, exhibiendo una variabilidad de rendimiento muy pronunciada entre las diferentes consultas.

En la Tabla I se observa que las consultas 7, 8, 9, 12, 18 y 20 alcanzaron una Precisión Media perfecta ($AP = 1.0000$). Esto se debe a que contienen términos muy específicos y poco frecuentes en el corpus (con un alto valor de frecuencia inversa de documento o IDF), como “*brillantes*”, “*decepción*”, “*maestra*”, o “*western*”. Al introducir estos términos discriminatorios, el modelo vectorial pondera fuertemente esas dimensiones, situando los documentos relevantes al inicio del ranking.

Por el contrario, los peores resultados se registraron en las siguientes necesidades de información:

- **Necesidad 19** ($AP = 0.2685$): “*Que sea de aventura y fantasía en inglés, y sea visualmente increíble con guion que flojea*”. Al ser una consulta larga con términos muy comunes (“*aventura*”, “*fantasía*”, “*inglés*”), los términos específicos determinantes de la relevancia (como “*flojea*”) ven diluida su influencia en el cálculo de la similitud del coseno, permitiendo que películas no deseadas superen a las deseadas en el ranking.
- **Necesidad 4** ($AP = 0.2766$): “*Que no sea ciencia ficción y sea de aventura, y no sea predecible*”. La consulta fue “*aventura no ciencia ficción no predecible*”. Aquí se hace patente la incapacidad del modelo vectorial basado en bolsas de palabras para procesar negaciones lógicas. La palabra “no” suele filtrarse como stop word, por lo que el sistema evalúa la consulta como si el usuario buscara documentos relacionados con “*ciencia*”, “*ficción*” y “*predecible*”, devolviendo resultados opuestos a la necesidad real.
- **Necesidad 2** ($AP = 0.4289$): “*Que sea de fantasía y apta para todos los públicos*”. Al no contar con términos raros de alta selectividad, la similitud del coseno se vio afectada por el solapamiento de vocabulario común entre películas que no necesariamente cumplían con el criterio de relevancia familiar en sentido estricto.

D. Análisis Comparativo y Discusión

Al comparar ambos enfoques, se evidencian las ventajas y desventajas de cada paradigma:

- 1) **Gradación vs. Todo o Nada:** El modelo booleano no proporciona ningún ordenamiento de relevancia, clasifica el corpus de forma binaria. El modelo vectorial puntúa cada documento en una escala continua $[0, 1]$, permitiendo presentar al usuario un ranking ordenado de mayor a menor relevancia estimada, clave en colecciones grandes.

- 2) **Expresividad y Usabilidad:** Las consultas booleanas requieren un esfuerzo cognitivo elevado y conocimientos de lógica por parte del usuario. El modelo vectorial acepta texto libre y tolera correspondencias parciales de forma flexible.
- 3) **Manejo de la Negación:** El modelo booleano maneja con precisión la exclusión de términos mediante el operador NOT. En cambio, el modelo vectorial básico basado en similitud de coseno no distingue negaciones, lo que penaliza su desempeño en consultas con exclusiones explícitas.

V. CONCLUSIONES

En el presente trabajo se ha diseñado, desarrollado y evaluado un sistema de búsqueda y recuperación cinematográfico, comparando el comportamiento del modelo booleano y del modelo de espacio vectorial. Para ello, se consolidó un corpus documental de 70 películas unificando metadatos oficiales de TMDb y críticas de usuarios de IMDb. Un aspecto clave del desarrollo fue la adaptación del contenido al español. Tras un preprocesamiento lingüístico clásico, la información se indexó estructuradamente en la biblioteca Whoosh. Los sistemas se evaluaron cuantitativamente mediante un banco de 20 necesidades de información representativas, arrojando resultados promediados algo dispares: el modelo booleano alcanzó una precisión de 0.9980 y un recall de 1.0000, mientras que el modelo vectorial obtuvo un Mean Average Precision (MAP) de 0.7248.

Un análisis detenido del rendimiento revela importantes vulnerabilidades y limitaciones operativas en ambas aproximaciones. En primer lugar, la aparente infalibilidad del modelo booleano está fuertemente sesgada por el acoplamiento cognitivo de las autoras al diseñar las necesidades de información. En un escenario real, este modelo se ve limitado por la rigidez del problema del desacoplamiento de vocabulario (sinonimia y polisemia). El usuario común se ve obligado a conocer el léxico exacto de la base de datos para evitar recuperar demasiados documentos o ninguno. Además, hemos comprobado que la indexación de un campo unificado de texto introduce ruido lógico: el sistema booleano es vulnerable a falsos positivos cuando términos clave de la consulta aparecen en contextos descriptivos ajenos a su función real (como ocurrió al clasificar a *The Hobbit: The Desolation of Smaug* como una película de acción simplemente porque su sinopsis mencionaba que estaba “cargada de acción”, a pesar de no pertenecer de forma explícita a ese género en sus metadatos).

Por su parte, el modelo vectorial basado en TF-IDF y similitud de coseno, aunque ofrece una experiencia mucho más flexible al ordenar los resultados por relevancia y admitir texto libre, presenta fallos de diseño estructurales ante consultas complejas. Su mayor vulnerabilidad es la incapacidad para gestionar la negación lógica. Al filtrar palabras como “no” o “sin” por considerarlas palabras vacías (stop words), el motor acaba puntuando positivamente los documentos que contienen precisamente los términos que el usuario deseaba excluir, devolviendo resultados opuestos a la intención de búsqueda.

TABLA I
RESULTADOS DETALLADOS POR NECESIDAD DE INFORMACIÓN (NI) PARA AMBOS SISTEMAS

NI	Descripción resumida de la Necesidad de Información	Precisión Booleana	Recall Booleano	AP Vectorial (Coseno)
1	Ciencia ficción, no romántica	1.00	1.00	0.7649
2	Fantasia, familiar o animación	1.00	1.00	0.4289
3	Inglés, fantasía, recomendada	1.00	1.00	0.8752
4	Aventura, no ciencia ficción, no predecible	1.00	1.00	0.2766
5	Animación familiar, no reflexiva	1.00	1.00	0.6679
6	Acción en español, no predecible	0.96	1.00	0.6075
7	Ciencia ficción, aventura, visualmente increíble	1.00	1.00	1.0000
8	Comedia de animación, no decepcionante	1.00	1.00	1.0000
9	Drama en inglés, actuaciones brillantes	1.00	1.00	1.0000
10	Suspense y crimen, no lenta	1.00	1.00	0.5429
11	Familiar en español, obra maestra	1.00	1.00	0.5000
12	Drama romántico en inglés, joya oculta	1.00	1.00	1.0000
13	Acción, ciencia ficción, decepción	1.00	1.00	0.9500
14	Aventura, fantasía en español, joya oculta	1.00	1.00	0.5000
15	Aventura en español, obra maestra	1.00	1.00	0.8542
16	Aventura en inglés, entretenida y predecible	1.00	1.00	0.8100
17	Acción, ciencia ficción en español, reflexión profunda	1.00	1.00	0.4500
18	Suspense en inglés, fascinante, bien estructurada	1.00	1.00	1.0000
19	Aventura, fantasía en inglés, visualmente increíble, guion flojo	1.00	1.00	0.2685
20	Acción en inglés, drama/western, joya oculta	1.00	1.00	1.0000
Promedio Global (Métricas del Sistema)		0.9980	1.0000	0.7248 (MAP)

Asimismo, el modelo es susceptible a la dilución de términos en consultas extensas, donde el solapamiento de palabras muy frecuentes en el corpus (con bajo valor IDF, como “aventura” o “inglés”) empaña y desplaza en el ranking a los descriptores específicos y escasos (con alto valor IDF, como “flojea”), penalizando la precisión media del sistema.

Como trabajo futuro, se plantean diversas líneas de mejora orientadas a mitigar estas vulnerabilidades. La primera de ellas consiste en la implementación de un motor de búsqueda híbrido, en el cual se aplique una preselección booleana estricta (para realizar filtrados de metadatos exactos y exclusiones mediante negación lógica) antes de computar la matriz TF-IDF y ordenar los documentos supervivientes mediante la similitud del coseno. En segundo lugar, se propone el uso de técnicas de búsqueda basadas en el significado (búsqueda semántica) para que el sistema intente comprender el sentido real y el contexto de las consultas en lugar de limitarse a buscar palabras exactas. Esto ayudaría a que el buscador entienda sinónimos de manera natural y gestione correctamente las frases con negaciones, superando las discrepancias lógicas y léxicas que surgen al comparar palabras de forma aislada. Finalmente, a nivel práctico, se proyecta el desarrollo de una interfaz web o gráfica de usuario que reemplace la terminal de comandos actual, ofreciendo una experiencia interactiva más atractiva para el usuario final.

REFERENCIAS

- [1] C. D. Manning, P. Raghavan, y H. Schütze, *Introduction to Information Retrieval*. Cambridge, Reino Unido: Cambridge University Press, 2008.
- [2] Á. Romero Jiménez, “Propuesta de trabajo práctico de la asignatura Inteligencia Artificial: Recuperación de la información,” Dpto. de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla, Curso 2025/2026.
- [3] S. Bird, E. Klein, y E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, EE. UU.: O’Reilly Media, 2009.

- [4] The Movie Database (TMDB), “TMDB 5000 Movie Dataset,” Kaggle, 2017. [En línea]. Disponible en: <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>
- [5] V. D. Pham, “IMDB Dataset,” Kaggle, 2019. [En línea]. Disponible en: <https://www.kaggle.com/code/phamvudung/imdb-dataset>
- [6] Google, “Gemini” (Modelo de lenguaje a gran escala), 2026. [En línea]. Disponible en: <https://gemini.google.com>